

Extract Text from a PDF

You can extract text from a PDF:

```
from pypdf import PdfReader

reader = PdfReader("test Orient.pdf")
page = reader.pages[0]
print(page.extract_text())

# extract only text oriented up
print(page.extract_text(0))

# extract text oriented up and turned left
print(page.extract_text((0, 90)))

# extract text in a fixed width format that closely adheres to the rendered
# layout in the source pdf
print(page.extract_text(extraction_mode="layout"))

# extract text preserving horizontal positioning without excess vertical
# whitespace (removes blank and "whitespace only" lines)
print(page.extract_text(extraction_mode="layout", layout_mode_space_vertically=False))

# adjust horizontal spacing
print(page.extract_text(extraction_mode="layout", layout_mode_scale_weight=1.0))

# exclude (default) or include (as shown below) text rotated w.r.t. the page
print(page.extract_text(extraction_mode="layout", layout_mode_strip_rotated=False))
```

Refer to `extract_text()` for more details.

! Note

Extracting the text of a page requires parsing its whole content stream. This can require quite a lot of memory - we have seen 10 GB RAM being required for an uncompressed content stream of about 300 MB (which should not occur very often).

To limit the size of the content streams to process (and avoid OOM errors in your application), consider checking `len(page.get_contents().get_data())` beforehand.

If a PDF page appears to contain only an image (e.g., a scanned document), the extracted text may be minimal or visually empty. In such cases, consider using OCR software such as [Tesseract OCR](#) to extract text from images.

Using a visitor

You can use visitor functions to control which part of a page you want to process and extract. The visitor functions you provide will get called for each operator or for each text fragment.

The function provided in argument `visitor_text` of function `extract_text` has five arguments:

- `text`: the current text (as long as possible, can be up to a full line)
- `user_matrix`: current matrix to move from user coordinate space (also known as CTM)
- `tm_matrix`: current matrix from text coordinate space
- `font_dictionary`: full font dictionary
- `font_size`: the size (in text coordinate space)

The matrix stores six parameters. The first four provide the rotation/scaling matrix, and the last two provide the translation (horizontal/vertical). It is recommended to use the `user_matrix` as it takes into account all transformations.

Notes :

- As indicated in §8.3.3 of the PDF 1.7 or PDF 2.0 specification, the user matrix applies to text space/image space/form space/pattern space.
- If you want to get the full transformation from text to user space, you can use the `mult()` function as follows: `txt2user = mult(tm, cm)`. The font size is the raw text size and affected by the `user_matrix`.

The `font_dictionary` may be `None` in case of unknown fonts. If not `None`, it could contain something like the key `"/BaseFont"` with the value `"/Arial,Bold"`.

Caveat: In complicated documents, the calculated positions may be difficult to determine (if you move from multiple forms to page user space, for example).

The function provided in argument `visitor_operand_before` has four arguments: operator, operand-arguments, current transformation matrix, and text matrix.

Example 1: Ignore header and footer

The following example reads the text of page four of [this PDF document](#), but ignores the header ($y > 720$) and footer ($y < 50$). In this file we also need to include new line characters ($y == 0$).

```
from pypdf import PdfReader

reader = PdfReader("GeoBase_NHNC1_Data_Model_UML_EN.pdf")
page = reader.pages[3]

parts = []

def visitor_body(text, cm, tm, font_dict, font_size):
    y = tm[5]
    if 50 < y < 720 or y == 0:
        parts.append(text)

page.extract_text(visitor_text=visitor_body)
text_body = "".join(parts)

print(text_body)
```

Example 2: Extract rectangles and texts into an SVG file

The following example converts page three of [this PDF document](#) into an [SVG file](#).

Such an SVG export may help to understand what is going on in a page.

```

from pypdf import PdfReader
import svgwrite

reader = PdfReader("GeoBase_NHNC1_Data_Model_UML_EN.pdf")
page = reader.pages[2]

dwg = svgwrite.Drawing("GeoBase_test.svg", profile="tiny")

def visitor_svg_rect(op, args, cm, tm):
    if op == b"re":
        (x, y, w, h) = (args[i].as_numeric() for i in range(4))
        dwg.add(dwg.rect((x, y), (w, h), stroke="red", fill_opacity=0.05))

def visitor_svg_text(text, cm, tm, font_dict, font_size):
    (x, y) = (cm[4], cm[5])
    dwg.add(dwg.text(text, insert=(x, y), fill="blue"))

page.extract_text(
    visitor_operand_before=visitor_svg_rect, visitor_text=visitor_svg_text
)
dwg.save()

```

The SVG generated here is bottom-up because the coordinate systems of PDF and SVG differ.

Unfortunately, in complicated PDF documents the coordinates given to the visitor functions may be wrong.

Why Text Extraction is hard

Unclear Objective

Extracting text from a PDF can be tricky. In several cases, there is no clear answer to what the expected result should look like:

1. **Paragraphs:** Should the text of a paragraph have line breaks at the same places where the original PDF had them or should it rather be one block of text?
2. **Page numbers:** Should they be included in the extract?
3. **Headers and Footers:** Similar to page numbers - should they be extracted?
4. **Outlines:** Should outlines be extracted at all?
5. **Formatting:** If the text is **bold** or *italic*, should it be included in the output?
6. **Tables:** Should the text extraction skip tables? Should it extract just the text? Should the borders be shown in some Markdown-like way or should the structure be n

8. **Ligatures:** The Unicode symbol [U+FB00](#) is a single symbol ff for two lowercase letters 'f'. Should that be parsed as the Unicode symbol 'ff' or as two ASCII symbols 'ff'?
9. **SVG images:** Should the text parts be extracted?
10. **Mathematical Formulas:** Should they be extracted? Formulas have indices and nested fractions.
11. **Whitespace characters:** How many new lines should be extracted for 3 cm of vertical whitespace? How many spaces should be extracted if there is 3 cm of horizontal whitespace? When would you extract tabs and when spaces?
12. **Footnotes:** When the text of multiple pages is extracted, where should footnotes be shown?
13. **Hyperlinks and Metadata:** Should it be extracted at all? Where should it be placed in which format?
14. **Linearization:** Assume you have a floating figure in between a paragraph. Do you first finish the paragraph, or do you put the figure text in between?

Then there are issues where most people would agree on the correct output, but the way PDF stores information just makes it hard to achieve that:

1. **Tables:** Typically, tables are just absolutely positioned text. In the worst case, every single letter could be absolutely positioned. That makes it hard to tell where columns / rows are.
2. **Images:** Sometimes PDFs do not contain the text as it is displayed, but instead an image. You notice that when you cannot copy the text. Then there are PDF files that contain an image and a text layer in the background. That typically happens when a document was scanned. Although the scanning software (OCR) is pretty good today, it still fails once in a while. pypdf is no OCR software; it will not be able to detect those failures. pypdf will also never be able to extract text from images.

Finally, there are issues that pypdf will deal with. If you find such a text extraction bug, please share the PDF with us so we can work on it!

Missing Semantic Layer

The PDF file format is all about producing the desired visual result for printing. It was not created for parsing the content. PDF files don't contain a semantic layer.

Specifically, there is no information what the header, footer, page numbers, tables, and paragraphs are. The visual appearance is there, and people might find heuristics to make educated guesses, but there is no way of being certain.

This is a shortcoming of the PDF file format, not of pypdf.

It is possible to apply machine learning on PDF documents to make good heuristics, but that will not be part of pypdf. However, pypdf could be used to feed such a machine learning system with the relevant information.

Whitespaces

The PDF format is meant for printing. It is not designed to be read by machines. The text within a PDF document is absolutely positioned, meaning that every single character could be positioned on the page.

The text

This is a test document by Ethan Nelson.

can be represented as

```
[(This is a )9(te)-3(st)9( do)-4(cu)13(m)-4(en)12(t )-3(b)3(y)-3( )9(Et)-2(h)3(an)4(Nels)13(o)-5(n)3(.)] TJ
```

Where the numbers are adjustments of vertical space. This representation used within the PDF file makes it very hard to guarantee correct whitespaces.

More information:

- [issue #1507](#)
- [Negative numbers in PDF content stream text object](#)
- Mark Stephens: [Understanding PDF text objects](#), 2010.

OCR vs. Text Extraction

Optical Character Recognition (OCR) is the process of extracting text from images. Software which does this is called *OCR software*. The [tesseract OCR engine](#) is the most commonly known open source OCR software.

pypdf is **not** OCR software.

Digitally-born vs. Scanned PDF files

PDF documents can contain images and text. PDF files don't store text in a sen

If you scan a document, the resulting PDF typically shows the image of the scan. Scanners then also run OCR software and put the recognized text in the background of the image. pypdf can extract this result of the scanners OCR software. However, in such cases, it's recommended to directly use OCR software as errors can accumulate: The OCR software is not perfect in recognizing the text. Then it stores the text in a format that is not meant for text extraction and pypdf might make mistakes parsing that.

Hence, I would distinguish three types of PDF documents:

- **Digitally born PDF files:** The file was created digitally on the computer. It can contain images, texts, links, outline items (a.k.a., bookmarks), JavaScript, ... If you Zoom in a lot, the text still looks sharp.
- **Scanned PDF files:** Any number of pages was scanned. The images were then stored in a PDF file. Hence, the file is just a container for those images. You cannot copy the text, you don't have links, outline items, JavaScript.
- **OCRed PDF files:** The scanner ran OCR software and put the recognized text in the background of the image. Hence, you can copy the text, but it still looks like a scan. If you zoom in enough, you can recognize pixels.

Can we just always use OCR?

You might now wonder if it makes sense to just always use OCR software. If the PDF file is digitally-born, you can render it to an image.

I would recommend not to do that.

Text extraction software like pypdf can use more information from the PDF than just the image. It can know about fonts, encodings, typical character distances and similar topics.

That means pypdf has a clear advantage when it comes to characters which are easy to confuse such as `ooö`. **pypdf will never confuse characters.** It just reads what is in the file.

pypdf also has an edge when it comes to characters which are rare, e.g. 🤪. OCR software will not be able to recognize smileys correctly.

Attempts to prevent text extraction

If people who share PDF documents want to prevent text extraction, they have multiple ways to do so:

However, text extraction cannot be completely prevented if people should still be able to read the document. In the worst case, people can make a screenshot, print it, scan it, and run OCR over it.